Bug Investigation Agent

Manager Meeting Briefing

Quick reference for interview / deep-dive. Read on mobile while discussing.

Stack: .NET 8 Blazor Server | Custom orchestrator | Playwright | Optional LLM

OPENING (30 seconds)

Say: "This is a custom .NET investigation agent - not LangChain or Semantic Kernel. It runs a fixed pipeline of skills: Jira -> environment -> browser login -> reproduce -> (optional LLM investigate) -> human approval -> apply -> validate -> retest. V1 does NOT auto-commit or open PRs. Human stays in control."

1. AGENT FRAMEWORK

Using a framework?

No. No LangChain / Semantic Kernel / AutoGen.

Why not?

We needed a controlled engineering workflow with permissions, credentials, and a human gate - not free-form tool loops. Custom keeps V1 predictable and reviewable.

Orchestration custom?

Yes. AgentOrchestrator runs skills in a hardcoded order.

Framework later?

Maybe - for adaptive planning / multi-tool reasoning. Not required for the current pipeline.

Say: "Orchestration is intentional and sequential. The LLM is a specialist, not the boss."

2. SKILLS

What is a skill?

A permission-gated capability (ISkill) that does one job: Jira fetch, browser login, reproduce, investigate code, apply diff, validate, report evidence.

How does it know which skill?

The orchestrator picks them by id in order - NOT an LLM router.

Active skills (pipeline order)

- Jira
- Environment discovery
- Browser login
- Reproduction
- Code investigation (LLM)
- Code modification (after approval)
- Validation
- Evidence reporting

Disabled stubs in V1

- Git, GitHub, Pull Request, CI/CD, Deployment

How much KMS knowledge is in skills?

Skills are mostly GENERIC. KMS-shaped knowledge lives mainly in navigation/repro heuristics (TicketStepPlanner / ReproductionSkill): Modules vs Administration, Auditor Proposal, Investigator Checklists. Skills are NOT meant to hold all of KMS.

How new behavior is added

- Better ticket steps / rules in the planner
- Human-validated memory lessons (SQLite)
- Workspace rules (.cursor/rules, AGENTS.md) injected into LLM
- New skill classes registered in DI

Say: "Skills = tools. Product knowledge = layered heuristics + memory + repo rules - not one giant skill."

3. JIRA

Where ticket enters

User pastes ticket key/URL in the UI, then Analyze.

Authentication

Atlassian email + API token -> HTTP Basic (email:token).

How ticket is retrieved

GET /rest/api/3/issue/{key} (+ myself check, comments).

Summary / description

Used as context for discovery, defect description, and LLM prompt.

Reproduction steps

Parsed from description; drive the browser plan when present.

Environment / URLs

Extracted for portal login target.

Attachments / screenshots

Metadata only today - not downloaded into the browser run.

Settings live in Account / local settings: BaseUrl, email, API token.

4. NAVIGATION

How it knows where to go

From ticket text first (Steps to Reproduce / Admin -> X -> Y).

Entry point

Rules choose Administration vs Modules vs direct label - NOT always Modules.

Ticket -> plan

TicketStepPlanner expands steps / arrow paths into clickable steps.

Where implemented

TicketStepPlanner.cs + ReproductionSkill.TryFollowStepAsync

LLM or rules?

Deterministic RULES for navigation. LLM is NOT used for clicking.

New / unfamiliar area?

Relies on Jira steps + generic clicks; may end Ambiguous. Known V1 limit - adaptive/LLM planning is future work.

Say: "Navigation is rules-first on purpose so we can explain every click. LLM comes later for unfamiliar UIs."

5. BROWSER AUTOMATION

- Uses Microsoft Playwright (Chromium), headless.
- Navigation plan -> actions: click by role/label/text, open grids, Save/Promote, tabs, etc.
- Login: opens env URL, fills portal username/password (#UserName / #Password).
- Session: PlaywrightSessionHub keeps one browser/page per investigation so login survives across skills.
- Did click work?: page fingerprint (URL/title/heading); mark failed if UI did not change.
- Screenshots: after meaningful steps under App_Data/evidence/...
- Reproduced?: ticket-type signals (JS toast, confirmation text, empty field-wrapper for Text Response) - NOT LLM

vision on images.

Say: "Browser is Playwright with a shared session. Reproduction is evidence + heuristics, not AI guessed it."

6. AI / LLM

Where used?

Almost only CodeInvestigationSkill (root cause + proposed fix).

Providers?

OpenAI-compatible and Anthropic implemented; Azure/Cursor stubs; default can be null (offline).

Why provider-agnostic?

IAgentRuntime interface - swap provider without rewriting skills/orchestrator.

What is sent?

Ticket summary/description/steps/notes + keyword-matched code snippets + safety/workspace rules + memory lessons. Secrets are redacted.

Root cause / files / fix?

LLM returns structured JSON: rootCause, proposedFiles, proposedFixSummary, proposedDiff.

No API key?

Can still reproduce + collect evidence; completes without a real code fix.

Say: "LLM investigates code AFTER we reproduce. It does not drive Jira or browser navigation in V1."

7. MEMORY

What is it?

Human-validated engineering lessons (IEngineeringMemoryStore).

Where stored?

SQLite via Entity Framework.

How lessons added?

After successful human-validated outcomes (Memory UI).

How retrieved?

Filter by workspace + simple tag/text match, then inject into LLM prompt.

Is this RAG?

No - no embeddings / vector search yet.

Smarter over time?

Add embeddings/RAG, link lessons to module/ticket patterns, feed failed navigation into lessons.

8. CODE CHANGES

Can it fix a bug?

It can PROPOSE a fix; apply only after human approval.

Modify source automatically?

Only if human Approves -> CodeModificationSkill applies a local diff.

Commit / push / PR?

No in V1 (skills stubbed; permissions off).

How prevent uncontrolled changes?

Permissions gate, approval waiter, allowlisted validation commands, secret redaction, no git push.

Final success intent: local changes validated -> ready for human review.

9. COMPLEX BUGS (be honest)

Today

- Strong at: structured tickets, clear steps, UI defects with clear signals.
- Weak at: deep multi-hop bugs, wrong menu guesses, incomplete steps.

Capability checklist

Correlate browser <-> code

Partially (repro notes + LLM on workspace).

Inspect logs

No (not in V1).

Trace UI -> API -> backend -> DB

No (not in V1).

Identify root cause

Via LLM + local code search after successful reproduce.

Propose + validate fix

Yes path: propose -> approve -> apply -> dotnet build/test -> browser retest.

Say: "V1 is reproduce + evidence + LLM code proposal. Full request tracing is roadmap, not current."

10. FUTURE DIRECTION

- Adaptive navigation: LLM/planner chooses clicks when rules fail; verify with DOM fingerprints.
- Learn from investigations: failed steps + successful paths -> memory (later RAG).
- Multi-project: already workspace-scoped; keep product knowledge in memory/rules, not hardcoded forever.
- More than ask LLM about Jira: we already open the env, click the app, capture evidence, gate code changes, and retest.
- Valuable engineering agent = reliable repro + evidence + safe patch + (later) PR + CI + log/API correlation.

IF HE DIGS THE CODE

Pipeline

AgentOrchestrator.cs

Skill contract

ISkill / SkillIds

Nav rules

TicketStepPlanner.cs

Browser actions

ReproductionSkill.cs, BrowserAutomationSkill.cs

Session hub

PlaywrightSessionHub

LLM

CodeInvestigationSkill.cs, IAgentRuntime

Memory

IEngineeringMemoryStore / EF store

Disabled git/PR

DisabledSkills.cs

TOUGH QUESTIONS - SHORT ANSWERS

Is it smart?

Say: "Smart enough to run an investigation pipeline; not a replacement for an engineer. Value is speed, consistency, and a safe human gate."

Why not just ChatGPT + Jira?

Say: "Because we actually open the env, click the app, capture evidence, gate code changes, and retest - not only summarize a ticket."

Why so much custom code?

Say: "Control: permissions, credentials, no surprise commits, explainable steps for demos and audits."

Biggest gap?

Say: "Navigation is still mostly rule-based; complex/unfamiliar UI paths and deep backend tracing need the next iteration."

CLOSING LINE

Say: "V1 proves the loop: ticket -> login -> reproduce -> evidence -> investigate -> approve -> validate -> retest, without giving the agent write access to git. Next we make navigation and learning adaptive so it handles more tickets with less hand-holding."

Tip: On mobile, open in landscape or zoom. Answer from architecture, stay honest on limits, point to the pipeline as the differentiator.